Agent Framework *w. Databricks*

Session 1

Conceptual foundations - 1

- Agent <-- (core concept; need not be AI)
- AI agent <-- (core concept; agent empowered with AI)
- Multi-agent systems <-- (later sessions)
- AI orchestration <-- (later sessions)
- AI orchestration platform <-- (later sessions)

Conceptual foundations - 2

- Agent <-- (core concept; need not be AI)
- AI agent <-- (core concept; agent empowered with AI)
- Multi-agent systems <-- (later sessions)
- AI orchestration <-- (later sessions)
- AI orchestration platform <-- (later sessions)

What makes an AI agent?

“AI agents are adaptive programs/systems designed to make decisions autonomously. In this context, autonomous decision-making specifically refers to not being dependent on human input/feedback to facilitate or validate its decision-making, at least within a range of conditions and circumstances.”

“Another way to define AI agents is as "compound software systems, inclusive of one or more AI models, that operate within an environment and take actions within it." (Feng et al, 2025; see below). In other words, combining this definition with the previous one, an AI agent is an AI system (e.g. a decision tree, a neural network, an LLM, etc.) that uses one or more tools to act and modify its environment autonomously.”

Ref: http://knightcolumbia.org/content

What AI agents can do

- Act to obtain information
- Create and execute workflows
- Perform actions and tasks
- Interact with:
  - External systems
  - Applications/tools
  - Other AI agents

Key concept: tool integration

- A tool is any resource that can be:
  - Reliably used to achieve a goal or a range of goals
  - Reliably accessed by an agent seeking to achieve said goals
- Examples:
  - Functions (e.g. error log query, send notification, financial computation)
  - Servers (e.g. wiki server, file server, email server)
  - Services (e.g. query engine, data processing, text translation)
  - Other models/agents (e.g. sentiment analysis model, RAG agent, etc.)
- Purpose for an agent: ***offer specialised capability reliably***
  — which the agent may not have on its own

Demo: *1-agent-loop*

Agents without tools are just models

- For agent to be “agentic”, it needs to be able to:
  - Interact with its environment
  - Plan and execute multi-step workflows
  - Access specific capabilities across its workflow — which could be long/indefinite
- Tool integration is key to all the above

Tools ground agents

- Statistical AI models e.g. ANNs and LLMs are non-deterministic
- They work essentially as:
  - Complex function approximators
  - Pattern matchers
- They are hence vulnerable to:
  - Context drift
  - Hallucinations
- Tools help avoiding the above issues

Topic: *Tool integration via MCP*

Motivating questions

How can an agent reliably access and obtain results from available tools?

How to ensure tools can be independently managed?

— especially relevant for tools that may:

- be changed for more suited/optimised versions
- be complex enough to require separate hosting and management

One popular solution MCP

Ref: MCP notes

"Model Context Protocol (MCP) is an open protocol that enables seamless integration between LLM applications and external data sources and tools. Whether you're building an AI-powered IDE, enhancing a chat interface, or creating custom AI workflows, MCP provides a standardized way to connect LLMs with the context they need."

Ref: https://modelcontextprotocol.io/specification/2026-07-28

“It standardises the procedure of requesting for context from a remote source and reading the context received from that remote source (here, "context" also refers to the results of tool calls); hence, the MCP protocol handles discovery, invocation, and response formatting.”

MCP architecture

MCP follows a client-server architecture where:

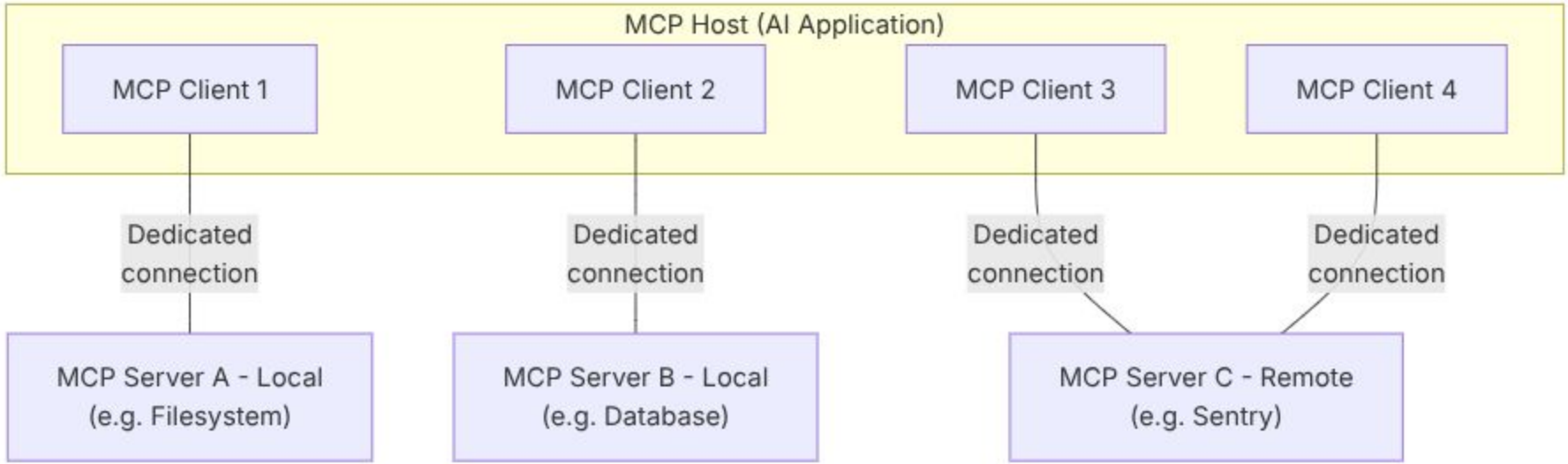
- 1 MCP host (an AI application) establishes connections to 1+ MCP servers
- By creating 1 MCP client for each MCP server

NOTE:

- Each MCP client maintains a dedicated connection with its MCP server
- A single host can connect to multiple servers simultaneously…
  … each through its own client instance

This lets an agent access tools from:
- UC functions
- external APIs
- custom services
… all through the same protocol.

Illustration



Ref: https://modelcontextprotocol.io/docs/2026-07-28/learn/architecture

Key participants in MCP’s architecture

- **MCP host**:
  The AI application that coordinates and manages one or multiple MCP clients
- **MCP client**:
  A component that:
  - Maintains a connection to an MCP server
  - Obtains context from an MCP server for the MCP host to use
- **MCP server**:
  A program that responds to MCP clients' requests

MCP lifecycle

```
Initialise | capability handshake
|
v
Discover   | via tools/list  | This is where:
|                            | - Tools are discovered
v                            | - Tools are called
Invoke     | via tools/call  | - Result/context is acquired
|
v
Shutdown   | close connection
```

Focusing on tool-based usage; usage for resources and prompts is similar.

Key conceptual details

See the following sections in :

- MCP's scope
- MCP's protocol layers
- MCP's statelessness
- MCP's primitives
- MCP lifecycle

Key takeaways

- Tools are discovered at runtime, not hardcoded
- MCP is an ecosystem built around a specification
- MCP’s communication standard combines 2 protocols
  — 1 in the data layer, 2 possible mechanisms in the transport layer
- The transport layer abstracts communication details from the data layer
- MCP is designed to be a stateless protocol
- MCP has 3 server-side primitives (tools, resources, prompts)
  — + 1 client-side primitive (elicitation)

Demo: *3-mcp*

Topic: *Observability via MLflow*

Motivating problem

- The flow within a complex distributed system can become opaque
  See: [Intent, pranigopu.github.io/getting-into-observability/otel.html](pranigopu.github.io/getting-into-observability/otel.html)
- However, agentic workflows need continuous evaluation/monitoring; why?
  - To combat context drift
  - To ensure the system adapts to changing/varying env's/requirements
    — especially relevant since such adaptability can't be 100% planned for
- Evaluation and monitoring need observability data, i.e.:
  - Metrics
  - Logs/events
  - Traces
    — the most relevant; see next slide

Traces as the core of detailed observability

Ref: [Trace, pranigopu.github.io/getting-into-observability/common-terms.html]

- A trace is a tree of spans
  — a span is a unit of work, i.e. captured info for a given block of time
- A trace is NOT simply a workflow's representation:
  - Parent span covers its child spans
  - A purely linear sequential workflow => breadth, not depth, i.e.:
    1 parent span (full workflow) + n child spans (linear steps)

"Traces (also known as distributed traces) are a type of data that captures requests as they flow through a system, capturing key steps across services and components. They help identify errors, slowdowns, and bottlenecks, allowing IT teams to resolve problems quickly and optimise performance."

How observability helps across agent development

Ref: Observability in agent deployment - notes

MLflow as the observability solution

Ref: MLflow notes

“MLflow is an open-source platform ("platform" being an infrastructure to host various related applications/functionalities) for managing lifecycle of ML/AI applications/deployments. This management includes: tracking specific applications/deployments (via ID'd "experiments" and training runs), deployment of ML/AI models, tracing ML/AI workflows, and evaluating outputs.”

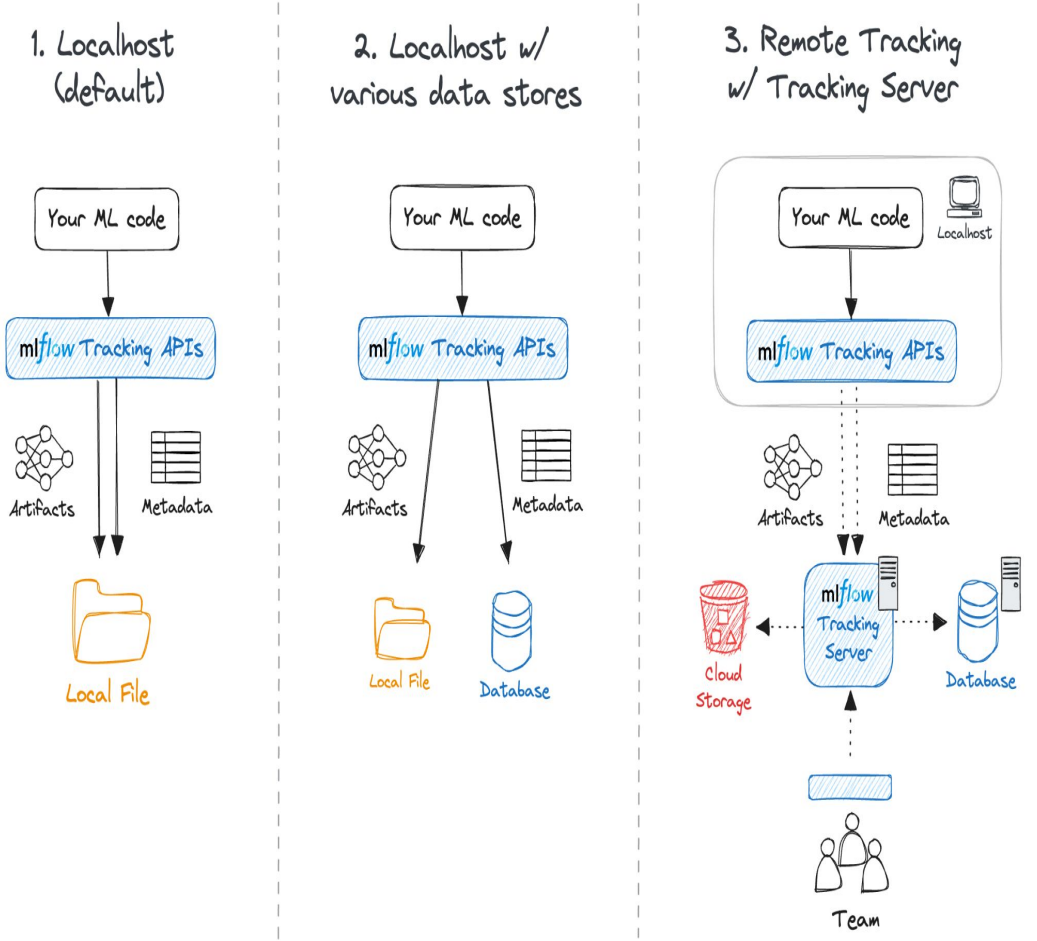
What MLflow covers

- Tracking (ML experiment tracking)
- Model versioning
- Model deployment
- Tracing
- Evaluation
- Monitoring (via continuous tracking, tracing, evaluation)

MLflow architecture

Ref: [MLflow architecture - notes]

Ref: [MLflow docs]



Briefly about MLflow's core components

- **MLflow Tracking Server**:
  "This is optional for local development since MLflow SDK can directly interact with a local database and file system. However, the server is essential for team development and provides features such as access control."
  - FastAPI server
  - That serves REST APIs
  - For accessing the backend and the artifact store
  - As well as hosting the MLflow UI
- **Backend Store**:
  A database or file system that stores metadata of experiments, runs, traces…
- **Artifact Store**:
  Store for artifacts for each run, such as model weights, images, model/data files
- **MLflow SDK**:
  Client SDKs in multiple languages with which users interact with the backend

Ref: https://www.mlflow.org/docs/latest/self-hosting/architecture/overview/

Demo: *local-mlflow-demo*

Demo: *2-tracing*

TOPIC: Agent evaluation

Motivating problem

Agentic workflows need continuous evaluation/monitoring; why?

- To combat context drift
- To ensure the system adapts to changing/varying env's/requirements — especially relevant since such adaptability can't be 100% planned for

Furthermore, evaluation cannot be limited to one stage of the agent's lifecycle:

- Development needs evaluation for iterative improvements
- Pre-deployment needs evaluation for assessing production-readiness
- Post-deployment needs evaluation to track drift or unexpected issues
- Data collected in production serves the previous levels

Reason for a separate paradigm for agentic AI

- Agent evaluation vs. traditional software testing
- Agent evaluation vs. AI model evaluation

Ref: Reason for a separate paradigm, agent evaluation - notes

Challenges in agent evaluation

- Multi-dimensional and multi-process complexity
- Safety and alignment
- Real-world variability
- Unstable/evolving quality signals

Ref: Challenges in agent evaluation, agent evaluation - notes

Demo: *4-evaluation*

Demo: *5-versions-loop*